

UNIVERSITE CLERMONT AUVERGNE
Clermont-Ferrand

Ecole Universitaire de Physique et d'Ingénierie

MEMOIRE DE STAGE

1^{ière} année de Master

Spécialité : Mécatronique

Nom et Prénom de l'étudiant

ABABSSA Haythem

**EXTRACTION DES MOUVEMENTS À PARTIR DES
VIDÉOS ÉGOCENTRIQUES
ENREGISTRÉES PAR DES ENFANTS**

Date de soutenance : 10/03/2026

Entreprise : Institut Pascal, Clermont Auvergne

REMERCIEMENTS

Je tiens tout d'abord à remercier l'ensemble de l'équipe *ComSee* de l'Institut Pascal pour son accueil, sa bienveillance et son soutien tout au long de la réalisation de ce Travail d'Étude et de Recherche. Je leur suis particulièrement reconnaissant de m'avoir offert l'opportunité d'intégrer leur environnement de recherche et de mener ce projet au sein de leur institut. Cette expérience m'a permis de découvrir le fonctionnement d'un laboratoire de recherche ainsi que les problématiques scientifiques actuelles liées à la vision par ordinateur et à l'intelligence artificielle.

J'adresse mes plus sincères remerciements à Monsieur **Arthur Aubret**, encadrant de ce projet, pour sa disponibilité, ses conseils avisés et la confiance qu'il m'a accordée durant toute la durée du TER. Son accompagnement, son expertise scientifique et la qualité de nos échanges ont constitué une aide précieuse dans la conduite de ce travail et dans l'approfondissement de mes connaissances.

Je remercie également le Mésocentre de l'Université Clermont Auvergne pour la mise à disposition des ressources de calcul ayant permis la réalisation des expérimentations présentées dans ce mémoire.

Enfin, je remercie l'École Universitaire de Physique et d'Ingénierie ainsi que l'Université Clermont Auvergne pour la qualité de la formation dispensée au cours de mon cursus, qui m'a permis d'acquérir les compétences scientifiques et techniques nécessaires à la réalisation de ce projet.

À toutes les personnes ayant contribué, directement ou indirectement, au bon déroulement de ce TER, j'adresse mes plus sincères remerciements.

Résumé

Ce Travail d'Étude et de Recherche porte sur l'extraction automatique d'informations géométriques comme la profondeur métrique et mouvement de caméra à partir de vidéos égocentriques enregistrées depuis le point de vue d'un enfant, à partir des vidéos BabyView.

Deux approches initiales ont été étudiées sur des séquences Ego4D : COLMAP pour l'estimation des poses caméra et Video Depth Anything pour l'estimation de profondeur. Ces méthodes ont révélé des limitations importantes, notamment l'absence d'échelle métrique réelle et un manque de robustesse sur les longues séquences.

La solution retenue repose sur le pipeline DroidMetric, combinant Metric3D pour la profondeur métrique et DROID-SLAM pour les poses caméra. Un pipeline automatisé a été développé et déployé sur le cluster HPC du Mésocentre UCA. Sa validation sur le dataset TUM RGB-D a confirmé la cohérence des trajectoires produites, avant son application aux vidéos BabyView, qui a permis de valider la méthode tout en révélant certaines limites ouvrant des perspectives d'amélioration.

Table des matières

Résumé	2
Table des figures	4
Liste des abréviations	6
Introduction	7
1 Présentation de la structure d'accueil	8
1.1 L'Institut Pascal	8
1.2 L'axe ISPR	8
1.3 L'équipe ComSee	9
1.4 Mon projet au sein de l'équipe	9
2 État de l'art	11
2.1 Vidéos égocentriques	11
2.2 Le projet BabyView	12
2.2.1 Dispositif de capture	12
2.2.2 Contenu et organisation du dataset	12
2.3 Apprentissage multimodal	13
2.4 Estimation de profondeur	13
2.5 Estimation du mouvement de la caméra	14
2.5.1 Visual Odometry	14
2.5.2 Visual SLAM	14
2.5.3 Méthodes basées sur l'apprentissage profond	14
3 Méthodologie	15
3.1 Approches initiales et leurs limites	15
3.1.1 Estimation des poses avec COLMAP	15
3.1.2 Estimation de profondeur avec Video Depth Anything	16
3.1.3 Bilan et motivation du choix	16
3.2 Solution retenue : le pipeline DroidMetric	16

3.2.1	Estimation de profondeur métrique avec Metric3D	16
3.2.2	Estimation des poses avec DROID-SLAM	17
3.3	Pipeline automatisé	17
3.3.1	Architecture et fonctionnement	17
3.3.2	Environnement d'exécution	18
3.3.3	Code source	18
4	Résultats expérimentaux	19
4.1	Protocole expérimental	19
4.1.1	Dataset TUM RGB-D	19
4.1.2	Séquences utilisées	19
4.1.3	Métriques d'évaluation	20
4.2	Validation sur les séquences TUM	20
4.3	Analyse des paramètres expérimentaux	21
4.3.1	Essais à 30 fps	22
4.3.2	Essais à 10 fps	22
4.3.3	Essais à 5 fps	23
4.3.4	Essais à 1 fps	24
4.4	Synthèse et discussion	24
4.4.1	Tableau récapitulatif	24
4.4.2	Impact de la calibration	25
4.4.3	Impact de la fréquence d'échantillonnage	25
4.4.4	Pertinence des métriques pour l'application visée	25
4.4.5	Analyse des erreurs résiduelles	26
4.4.6	Temps de calcul et passage à l'échelle	26
4.5	Validation qualitative sur les vidéos BabyView	27
4.5.1	Conditions d'application	27
4.5.2	Protocole de validation	27
4.5.3	Résultats et analyse	28
4.5.4	Bilan de la validation	29
4.6	Perspectives d'amélioration	30
4.6.1	Intégration d'un mécanisme de fermeture de boucle	30
4.6.2	Amélioration de la calibration intrinsèque	30
	Conclusion	31
	Annexe	32

Table des figures

2.1	Exemple de dispositif de capture BabyView : caméra GoPro portée sur le front de l'enfant [1].	12
3.1	Architecture du pipeline DroidMetric : de la vidéo aux poses caméra.	18
4.1	Trajectoires estimée et de référence — séquences checkerboard (gauche) et pioneer slam (droite).	20
4.2	Trajectoire estimée et de référence — translations pures (freiburg2_xyz). . .	21
4.3	Trajectoires estimée et de référence à 30 fps.	22
4.4	Trajectoires estimée et de référence à 10 fps.	23
4.5	Trajectoires estimée et de référence à 5 fps.	23
4.6	Trajectoires estimée et de référence à 1 fps.	24
4.7	Deux trajectoires BabyView issues de scènes sans déplacement global. La similarité visuelle entre les deux rend toute distinction impossible à l'œil nu.	28

Liste des abréviations

APE *Absolute Pose Error* : Erreur absolue de pose, mesure l'écart global entre la trajectoire estimée et la trajectoire de référence.

DBA *Dense Bundle Adjustment* : Optimisation dense des poses et des profondeurs utilisée dans DROID-SLAM.

fps *Frames per second* : Nombre d'images par seconde.

HPC *High Performance Computing* : Calcul haute performance. Désigne ici le cluster HPC2 du Mésocentre UCA.

RGB *Red Green Blue* : Espace colorimétrique standard des images couleur.

RGB-D *Red Green Blue Depth* : Image couleur accompagnée d'une carte de profondeur, fournie par des capteurs tels que la Microsoft Kinect.

RMSE *Root Mean Square Error* : Racine de l'erreur quadratique moyenne, indicateur de dispersion des erreurs utilisé conjointement à l'APE et à la RPE pour rendre compte de la sensibilité aux valeurs aberrantes.

RPE *Relative Pose Error* : Erreur relative de pose, mesure l'écart local entre déplacements successifs estimés et réels.

SfM *Structure from Motion* : Technique de reconstruction 3D à partir d'une séquence d'images.

SLAM *Simultaneous Localization and Mapping* : Localisation et cartographie simultanées.

TER Travail d'Étude et de Recherche.

VO *Visual Odometry* : Estimation du mouvement d'une caméra par analyse des images successives.

Introduction

Les progrès récents de l'apprentissage profond et de la vision par ordinateur ont permis le développement de modèles capables d'exploiter simultanément plusieurs types de données, ouvrant la voie à de nouvelles approches dites multimodales. Ces modèles combinent des sources d'information variées (images, vidéos, texte, audio) afin de construire des représentations plus riches du monde réel.

Dans ce cadre, les vidéos égocentriques constituent une source de données particulièrement pertinente. Filmées depuis le point de vue de la personne qui les porte, elles capturent les interactions naturelles d'un individu avec son environnement. Plusieurs travaux récents exploitent ce type de données pour étudier les mécanismes d'apprentissage visuel, notamment à partir de vidéos enregistrées par de jeunes enfants au cours de leurs premières années de vie.

Cependant, ces vidéos ne fournissent que des informations visuelles brutes. Des données géométriques essentielles, telles que la profondeur des scènes observées ou les déplacements de la caméra, ne sont pas directement disponibles. Or, ces informations sont nécessaires pour enrichir les jeux de données et faciliter leur exploitation dans un cadre d'apprentissage multimodal.

Ce Travail d'Étude et de Recherche s'inscrit dans cette problématique. Son objectif est d'étudier et de mettre en œuvre des méthodes permettant d'extraire automatiquement des informations de profondeur métrique et de mouvement à partir de vidéos égocentriques, dans le cadre du projet BabyView.

Ce mémoire est structuré en quatre chapitres. Le chapitre 1 présente la structure d'accueil de ce TER, à savoir l'Institut Pascal et l'équipe ComSee au sein de laquelle ce travail a été réalisé. Le chapitre 2 présente le contexte scientifique et un état de l'art des principales méthodes liées à l'estimation de profondeur et au mouvement de caméra. Le chapitre 3 décrit la méthodologie retenue et le pipeline développé. Le chapitre 4 présente les résultats expérimentaux obtenus sur le dataset TUM RGB-D et leur analyse. Une conclusion générale clôt le mémoire en dressant le bilan du travail accompli et en ouvrant des perspectives pour la suite du projet.

Chapitre 1

Présentation de la structure d'accueil

1.1 L'Institut Pascal

L'Institut Pascal est une unité mixte de recherche (UMR 6602) placée sous la tutelle principale de l'Université Clermont Auvergne (UCA) et du CNRS. C'est un laboratoire pluridisciplinaire spécialisé dans les sciences de l'ingénierie et des systèmes, dont les activités couvrent des domaines variés tels que l'imagerie, la robotique, l'intelligence artificielle, la mécanique, les matériaux, la photonique ou encore les technologies pour la santé. Il rassemble environ 400 personnes (chercheurs, enseignants-chercheurs, ingénieurs, doctorants et stagiaires) réparties sur le campus des Cézeaux et sur plusieurs sites annexes en région Auvergne.

Le laboratoire s'organise autour de cinq axes de recherche complémentaires :

- **GePEB** – Génie des Procédés, Énergétique et Biosystèmes
- **ISPR** – Image, Systèmes de Perception et Robotique
- **M3G** – Mécanique, Génie Mécanique, Génie Civil et Génie Industriel
- **PHOTON** – Photonique, Ondes et Nanomatériaux
- **TGI** – Thérapies Guidées par l'Image

Ces axes de recherche s'inscrivent dans une démarche interdisciplinaire, avec des applications qui s'articulent notamment autour de trois grandes thématiques : l'usine du futur, les transports du futur et l'hôpital du futur. C'est dans ce cadre que mon TER a été réalisé, au sein de l'axe ISPR.

1.2 L'axe ISPR

L'axe ISPR (Image, Systèmes de Perception et Robotique) regroupe des équipes travaillant sur la vision par ordinateur, la perception artificielle, la robotique, le traitement d'images, la reconstruction 3D, la navigation autonome ainsi que l'intelligence artificielle. L'objectif des recherches menées au sein de cet axe est de développer des systèmes capables de percevoir leur

environnement à partir d’images ou de données capteurs, puis d’analyser et d’interpréter ces informations pour prendre des décisions ou interagir de manière autonome avec leur environnement.

L’axe ISPR est composé de plusieurs équipes de recherche, parmi lesquelles l’équipe ComSee, au sein de laquelle j’ai effectué mon TER.

1.3 L’équipe ComSee

L’équipe ComSee (*Computers that See*), littéralement « les ordinateurs qui voient », est rattachée à l’axe ISPR de l’Institut Pascal. Son objectif principal est de développer des méthodes permettant aux systèmes informatiques d’analyser et d’interpréter automatiquement des images et des scènes complexes.

L’équipe organise ses activités de recherche autour de deux grandes thématiques complémentaires :

- la **vision géométrique**, qui traite de la modélisation et de l’étalonnage de capteurs de vision (caméras classiques, rolling-shutter, plénoptiques), ainsi que de la localisation et de la reconstruction 3D par vision ;
- l’**interprétation de séquences d’images**, qui vise à détecter, reconnaître et suivre les différents éléments d’une scène dans des vidéos dynamiques et complexes, y compris dans des conditions de visibilité dégradées, notamment à l’aide de techniques d’apprentissage et de transfert d’apprentissage.

À partir de ces deux axes, les travaux de l’équipe se déclinent plus concrètement autour de plusieurs activités, parmi lesquelles :

- l’étalonnage et la calibration de capteurs de vision embarqués ;
- la localisation et la cartographie simultanées (SLAM) ;
- la reconstruction 3D de scènes à partir de séquences vidéo ;
- le suivi et la reconnaissance d’objets ou de personnes en scène dynamique ;
- L’apprentissage profond appliqué à la perception visuelle.

Ces recherches trouvent des applications dans des domaines variés tels que les transports intelligents, la réalité augmentée, la vidéosurveillance et la vision robotique. L’équipe est composée d’enseignants-chercheurs, de chercheurs, de doctorants, de post-doctorants, d’ingénieurs et de stagiaires, offrant un environnement propice aux échanges scientifiques.

1.4 Mon projet au sein de l’équipe

J’ai réalisé mon TER au sein de l’équipe ComSee, sous la direction de M. Arthur Aubret. Ce travail s’inscrit directement dans les thématiques de l’équipe, en particulier la localisation

visuelle, la reconstruction 3D par vision et l'apprentissage profond appliqué à la perception visuelle.

Le suivi régulier assuré par mon encadrant m'a permis d'orienter mes choix méthodologiques et d'ajuster le protocole expérimental en fonction des contraintes rencontrées. Les traitements ont été menés sur les ressources GPU du Mésocentre de l'Université Clermont Auvergne, indispensables à l'exécution des modèles d'apprentissage profond mobilisés dans ce projet.

Chapitre 2

État de l’art

Ce chapitre présente les concepts et travaux qui constituent le cadre scientifique de ce TER. Le travail réalisé consiste à analyser des vidéos égocentriques afin d’en extraire des informations géométriques complémentaires. Ces informations, absentes des vidéos brutes, sont extraites dans l’optique d’être exploitées comme modalités supplémentaires au sein d’un futur apprentissage multimodal, qui constitue le cadre applicatif plus large dans lequel s’inscrit ce projet.

2.1 Vidéos égocentriques

Les vidéos égocentriques sont enregistrées du point de vue d’une personne, à l’aide d’une caméra portée sur la tête ou sur le corps. Contrairement aux vidéos filmées par un observateur externe, elles capturent ce que voit réellement un individu lors de ses activités quotidiennes. Ce type de données est exploité en vision par ordinateur, en robotique et en sciences cognitives, car il offre une représentation proche de l’expérience visuelle humaine [2].

Plusieurs jeux de données à grande échelle ont été constitués pour étudier la compréhension d’activités et la reconnaissance d’objets en contexte réel. Parmi les plus utilisés figurent EPIC-KITCHENS [3] et Ego4D [2], qui regroupent des milliers d’heures de vidéos capturées dans des situations quotidiennes variées.

Plus récemment, des travaux ont porté sur le développement cognitif de l’enfant à partir de vidéos égocentriques. L’idée est de soumettre des modèles d’apprentissage artificiel à un environnement visuel similaire à celui observé par les jeunes enfants, afin de mieux comprendre les mécanismes d’acquisition des connaissances. SAYCam [4], proposé par Sullivan et al., est l’un des premiers jeux de données de ce type.

C’est dans cette continuité que s’inscrit le présent travail, dont le dataset principal est présenté dans la section suivante.

2.2 Le projet BabyView

Le projet BabyView [1], proposé par Long et al., est un jeu de données de vidéos égocentriques collectées depuis le point de vue de jeunes enfants. L’objectif est de constituer une base permettant d’étudier les conditions visuelles dans lesquelles les enfants perçoivent et apprennent à interpréter leur environnement durant leurs premières années de vie.

2.2.1 Dispositif de capture

Les vidéos sont enregistrées à l’aide d’une caméra GoPro fixée sur un bandeau porté sur le front de l’enfant, comme illustré en figure 2.1. Ce dispositif capture ainsi le champ visuel naturel de l’enfant lors de ses activités quotidiennes à domicile.



FIGURE 2.1 – Exemple de dispositif de capture BabyView : caméra GoPro portée sur le front de l’enfant [1].

2.2.2 Contenu et organisation du dataset

Le dataset BabyView contient plusieurs centaines d’heures de vidéos enregistrées auprès de familles volontaires. Chaque séquence correspond à une session d’enregistrement d’un enfant lors de ses activités du quotidien : jeux, repas, interactions avec les parents ou exploration de l’environnement domestique. Les vidéos sont filmées en intérieur dans des conditions d’éclairage naturelles et variables.

Les données sont organisées par enfant et par session. Chaque fichier vidéo est au format .mp4, avec une résolution de 1920×1080 pixels et une fréquence de 30 fps. Les séquences

sont généralement comprises entre 10 et 20 minutes, ce qui représente un volume de données nettement supérieur aux séquences courtes utilisées dans les benchmarks classiques de Visual SLAM.

2.3 Apprentissage multimodal

L'apprentissage multimodal regroupe les méthodes permettant à un modèle d'exploiter simultanément plusieurs types de données, appelées modalités : images, vidéos, texte, audio, profondeur. L'objectif est d'apprendre des représentations communes à partir de sources d'information complémentaires, afin d'améliorer la compréhension du monde réel.

Dans ce contexte, les travaux d'Aubret et al. [5] s'inscrivent directement dans la problématique de ce TER. Les auteurs explorent comment les actions réalisées lors d'interactions avec des objets depuis un point de vue égo-centrique peuvent enrichir l'apprentissage auto-supervisé de représentations visuelles. Ils montrent que l'extraction des mouvements de caméra associés à ces interactions améliore la qualité des représentations apprises, ce qui souligne l'importance des informations géométriques (poses caméra et profondeur) pour les futures étapes d'apprentissage multimodal sur les vidéos BabyView.

L'exploitation de vidéos égo-centriques dans un cadre multimodal représente un axe de recherche actif. C'est précisément dans cette perspective que s'inscrit le pipeline développé dans ce TER.

2.4 Estimation de profondeur

L'estimation de profondeur consiste à associer à chaque pixel d'une image une valeur de distance entre la caméra et la surface correspondante, permettant ainsi de passer d'une représentation 2D à une description 3D de l'environnement.

Traditionnellement, cette information est obtenue à l'aide de capteurs dédiés tels que les caméras RGB-D ou les capteurs LiDAR. Lorsque ces dispositifs ne sont pas disponibles, des méthodes d'estimation à partir d'images classiques sont nécessaires. Parmi les travaux de référence figure MiDaS [6], qui a démontré une bonne capacité de généralisation sur des environnements variés grâce à un entraînement sur plusieurs jeux de données.

Une distinction fondamentale sépare deux types de résultats. La **profondeur relative** préserve les relations de distance entre les objets d'une scène, mais sans unité physique réelle. La **profondeur métrique** fournit des distances directement exprimées en mètres, ce qui est indispensable pour toute application quantitative. Cette distinction est centrale dans ce travail, car les méthodes étudiées initialement ne produisaient qu'une profondeur relative.

2.5 Estimation du mouvement de la caméra

L'estimation du mouvement de la caméra consiste à déterminer sa position et son orientation au cours du temps à partir d'une séquence d'images. Le résultat est une trajectoire composée de poses successives, représentées sous forme de matrices de transformation homogène 4×4 . Dans le cas des vidéos égocentriques, cette tâche est particulièrement délicate en raison des mouvements rapides, des changements d'éclairage fréquents et des zones pauvres en textures visuelles.

2.5.1 Visual Odometry

La Visual Odometry (VO) estime le déplacement d'une caméra en détectant et en suivant des points caractéristiques entre images successives [7]. Ces approches donnent de bons résultats sur des séquences courtes, mais souffrent d'une accumulation progressive des erreurs (phénomène appelé *dérive*) qui dégrade la précision sur les longues séquences.

2.5.2 Visual SLAM

Les systèmes de Simultaneous Localization and Mapping (SLAM) étendent la VO en construisant simultanément une carte de l'environnement. Des mécanismes de fermeture de boucle permettent de détecter lorsque la caméra repasse par un endroit déjà visité et de corriger les erreurs accumulées. ORB-SLAM2 [8], proposé par Mur-Artal et Tardós, est l'une des références les plus utilisées dans ce domaine.

2.5.3 Méthodes basées sur l'apprentissage profond

Les approches récentes intègrent l'apprentissage profond directement dans le processus d'estimation du mouvement. Plutôt que de détecter des points d'intérêt, ces méthodes apprennent des représentations visuelles à partir de grandes bases de données d'entraînement, ce qui leur confère une meilleure robustesse face aux mouvements rapides, aux variations d'éclairage et aux environnements peu texturés. Ces avantages sont particulièrement pertinents dans le contexte des vidéos égocentriques, et ont motivé le choix de la solution présentée au chapitre suivant.

Chapitre 3

Méthodologie

Ce chapitre présente la démarche suivie pour extraire automatiquement des informations géométriques à partir de vidéos égocentriques. Deux approches ont d’abord été étudiées sur des séquences du dataset Ego4D afin d’identifier leurs limites avant de retenir la solution finale. Le pipeline développé est ensuite décrit en détail.

3.1 Approches initiales et leurs limites

Les besoins du projet portent sur deux types d’informations : les poses de la caméra et les cartes de profondeur. Ces deux problématiques ont d’abord été abordées séparément à travers des méthodes existantes, testées sur des séquences vidéo du dataset Ego4D [2], avant tout accès aux données BabyView.

3.1.1 Estimation des poses avec COLMAP

COLMAP [9] est un outil de reconstruction tridimensionnelle basé sur la technique de Structure from Motion (SfM). Il estime les poses d’une caméra en détectant des points caractéristiques communs entre plusieurs images et en reconstruisant progressivement la structure 3D de la scène.

Les résultats obtenus sur les séquences Ego4D étaient partiellement satisfaisants : COLMAP parvenait à reconstruire des trajectoires cohérentes sur les portions de séquences comportant suffisamment de textures visuelles. Cependant, trois limitations importantes sont apparues.

Premièrement, les erreurs s’accumulent au fil du temps sur les longues séquences. Les approches SfM incrémentales comme COLMAP peuvent fragmenter la reconstruction en plusieurs composantes déconnectées et produire des trajectoires avec des facteurs d’échelle incohérents lorsque les séquences deviennent longues [10].

Deuxièmement, les mouvements rapides de caméra et les flous de mouvement fréquents dans les vidéos égocentriques provoquent des pertes de suivi. Ces systèmes échouent dans des conditions incluant peu de vues et du flou de mouvement [11].

Troisièmement, comme toute approche SfM, COLMAP ne fournit pas d'échelle métrique réelle : les poses estimées sont cohérentes entre elles, mais ne correspondent pas à des déplacements exprimés en mètres [9].

3.1.2 Estimation de profondeur avec Video Depth Anything

En parallèle, la méthode Video Depth Anything [12] a été appliquée aux mêmes séquences Ego4D pour générer des cartes de profondeur. Cette approche, basée sur l'apprentissage profond, produit des estimations denses et visuellement cohérentes, distinguant correctement les objets proches des éléments éloignés.

Toutefois, les profondeurs obtenues sont uniquement relatives : les valeurs estimées reflètent l'organisation spatiale de la scène mais ne correspondent pas à des distances réelles. Cette limitation est directement incompatible avec les objectifs du projet, qui requiert des informations géométriques quantitatives exploitables pour l'apprentissage multimodal.

3.1.3 Bilan et motivation du choix

Les deux approches répondent chacune partiellement aux besoins du projet : COLMAP fournit des poses caméra mais sans échelle métrique et avec une robustesse insuffisante sur les longues séquences, tandis que Video Depth Anything produit des cartes de profondeur de bonne qualité visuelle mais sans unité physique réelle. Ces observations ont conduit à rechercher une solution capable de fournir simultanément des poses fiables et des profondeurs métriques, adaptée aux contraintes des vidéos BabyView.

3.2 Solution retenue : le pipeline DroidMetric

La solution retenue repose sur le projet DroidMetric, qui combine deux méthodes complémentaires : Metric3D pour l'estimation de profondeur métrique et DROID-SLAM pour l'estimation des poses caméra. Ensemble, elles permettent d'extraire les deux types d'informations recherchées à partir d'une simple vidéo monoculaire.

Dans le cadre de ce TER, seuls les modules `depth.py` et `slam.py` ont été utilisés. Le module `mesh.py`, dédié à la reconstruction de maillages 3D, n'a pas été exploité car il ne répond pas aux objectifs du projet.

3.2.1 Estimation de profondeur métrique avec Metric3D

Metric3D [13], proposé par Yin et al., est un modèle d'apprentissage profond entraîné sur une grande variété de jeux de données couvrant des environnements et des caméras différents. Cette stratégie lui permet de généraliser à de nouvelles scènes sans recalibration, tout en produisant des estimations de profondeur exprimées directement en mètres.

À partir de chaque image RGB extraite de la vidéo, Metric3D prédit une carte de profondeur dense : chaque pixel se voit attribuer une valeur de distance en mètres entre la caméra et la surface correspondante. Ces cartes sont sauvegardées au format `.npy` et constituent l'entrée du module SLAM.

Contrairement aux approches étudiées précédemment (voir section 2.4), Metric3D produit une profondeur métrique réelle, ce qui est essentiel pour que les poses estimées par DROID-SLAM soient cohérentes à l'échelle physique de la scène.

3.2.2 Estimation des poses avec DROID-SLAM

DROID-SLAM [14], proposé par Teed et Deng, est un système de Visual SLAM basé sur l'apprentissage profond. Il traite les images d'une séquence sous la forme d'un graphe de vues : pour chaque paire d'images, un réseau de neurones extrait des correspondances visuelles qui sont ensuite utilisées pour contraindre les poses relatives.

L'optimisation des poses est réalisée de manière itérative par un module de Dense Bundle Adjustment différentiable, qui raffine simultanément les poses et les cartes de profondeur en exploitant l'ensemble des contraintes disponibles. Ce processus récurrent permet de corriger progressivement les erreurs accumulées, ce qui confère à DROID-SLAM une meilleure robustesse sur les longues séquences par rapport aux approches classiques.

Dans DroidMetric, les cartes de profondeur produites par Metric3D sont injectées comme initialisation dans SLAM. Cette initialisation métrique ancre l'optimisation dans une échelle physique réelle, ce qui évite l'ambiguïté d'échelle inhérente aux systèmes SLAM monoculaires classiques.

3.3 Pipeline automatisé

3.3.1 Architecture et fonctionnement

Le traitement d'une vidéo avec DroidMetric implique plusieurs étapes successives : extraction des frames, estimation des profondeurs, estimation des poses. Pour faciliter les expérimentations sur un grand nombre de séquences, ces étapes ont été regroupées dans un script unique, `reconstruct.py`, qui orchestre automatiquement l'ensemble du traitement à partir d'une seule commande.

Le schéma de la figure 3.1 illustre l'enchaînement des étapes. La vidéo en entrée est d'abord décomposée en images RGB à la fréquence d'échantillonnage spécifiée. Le module `depth.py` traite ensuite chaque image pour produire une carte de profondeur métrique au format `.npy`. Ces cartes sont transmises au module `slam.py`, qui les utilise comme initialisation pour estimer les poses de la caméra. Les résultats finaux poses et profondeurs sont enregistrés dans un répertoire dédié.

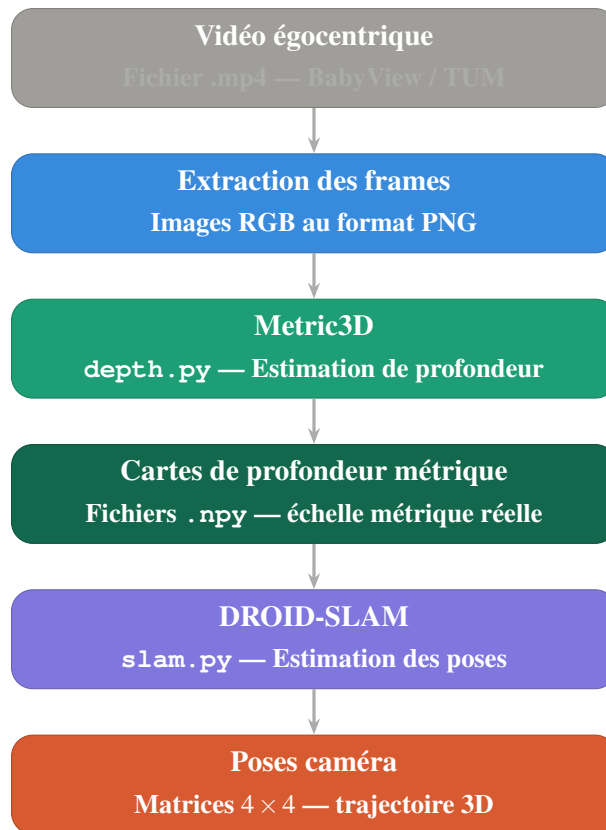


FIGURE 3.1 – Architecture du pipeline DroidMetric : de la vidéo aux poses caméra.

3.3.2 Environnement d'exécution

Les expérimentations ont été réalisées sur le cluster de calcul HPC2 du Mésocentre de l'Université Clermont Auvergne. Les jobs sont soumis via le gestionnaire SLURM et exécutés dans un conteneur Singularity contenant l'ensemble des dépendances du pipeline. Cette configuration permet de traiter plusieurs vidéos en parallèle et garantit la reproductibilité des expériences indépendamment de l'environnement local.

3.3.3 Code source

L'ensemble du pipeline utilisé au cours de ce TER est disponible dans le dépôt GitHub suivant :

https://github.com/aba631/Extraction_mvt

Ce dépôt a été constitué à partir du projet DroidMetric, dont seuls les modules `depth.py` et `slam.py` ont été conservés, le module `mesh.py` (dédié à la reconstruction de maillages 3D) ayant été retiré car il ne répondait pas aux besoins du projet. Le dépôt contient également `reconstruct.py` ainsi que les scripts auxiliaires nécessaires à la préparation des données et à l'évaluation des résultats.

Chapitre 4

Résultats expérimentaux

Ce chapitre présente l'évaluation quantitative et qualitative du pipeline DroidMetric. La méthode est d'abord validée sur trois séquences du dataset TUM RGB-D couvrant des types de mouvements variés. L'influence de la fréquence d'échantillonnage et de la calibration caméra est ensuite analysée sur une séquence dédiée.

4.1 Protocole expérimental

4.1.1 Dataset TUM RGB-D

Le dataset TUM RGB-D [15], proposé par Sturm et al., est un benchmark de référence pour l'évaluation des méthodes de Visual Odometry et de Visual SLAM. Il contient des séquences vidéo acquises avec une caméra Microsoft Kinect dans des environnements intérieurs, accompagnées d'une trajectoire de référence (*ground truth*) obtenue par un système de capture de mouvement infrarouge à haute précision. L'utilisation de ce dataset permet de comparer quantitativement les trajectoires estimées par DroidMetric aux trajectoires réelles, avant toute application aux vidéos BabyView pour lesquelles aucune vérité terrain n'est disponible.

4.1.2 Séquences utilisées

Trois séquences ont été sélectionnées pour la validation initiale du pipeline, couvrant des types de mouvements variés :

- `freiburg2_large_checkerboard_calibration` : mouvements lents face à une cible de calibration planaire ;
- `freiburg2_pioneer_slam2` : déplacements d'un robot mobile dans un couloir ;
- `freiburg2_xyz` : translations pures selon les trois axes.

L'analyse de l'influence des paramètres a été réalisée sur la séquence `freiburg2_desk_with_person`, qui présente des mouvements de caméra variés dans un environnement de bureau avec une personne en mouvement.

4.1.3 Métriques d'évaluation

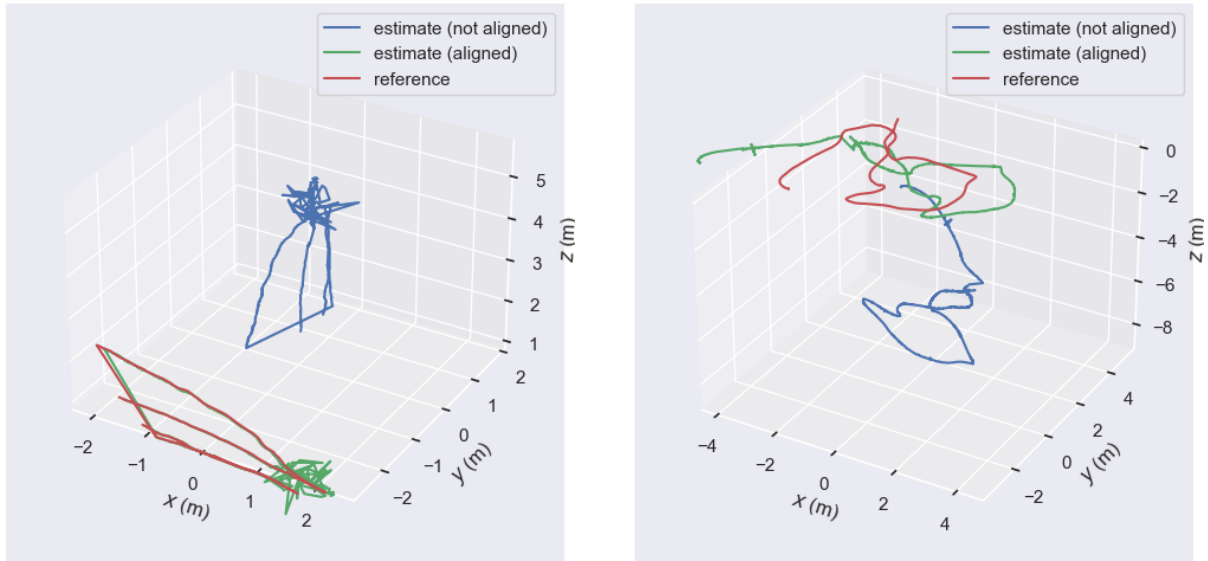
L'évaluation repose sur deux métriques calculées avec la bibliothèque EVO :

- **APE** (*Absolute Pose Error*) : écart en translation entre chaque pose estimée et la pose de référence correspondante. Elle quantifie la qualité globale de la trajectoire.
- **RPE** (*Relative Pose Error*) : erreur sur le déplacement entre deux poses consécutives. Elle évalue la précision locale de l'odométrie, indépendamment de l'alignement global.

Avant le calcul des métriques, les trajectoires estimée et de référence sont alignées par la méthode d'Umeyama [16], qui estime la transformation rigide (rotation et translation) minimisant l'écart quadratique moyen entre les deux ensembles de poses. La correction d'échelle n'est pas appliquée (`correct_scale=False`), car le pipeline produit des poses à échelle métrique réelle grâce à l'initialisation par Metric3D : corriger l'échelle masquerait les éventuelles erreurs d'estimation et fausserait l'évaluation.

4.2 Validation sur les séquences TUM

Les figures 4.1 et 4.2 présentent les trajectoires estimées comparées à la trajectoire de référence pour les trois séquences de validation.



(a) freiburg2_large_checkerboard_calibration

(b) freiburg2_pioneer_slam2

FIGURE 4.1 – Trajectoires estimée et de référence — séquences checkerboard (gauche) et pioneer slam (droite).

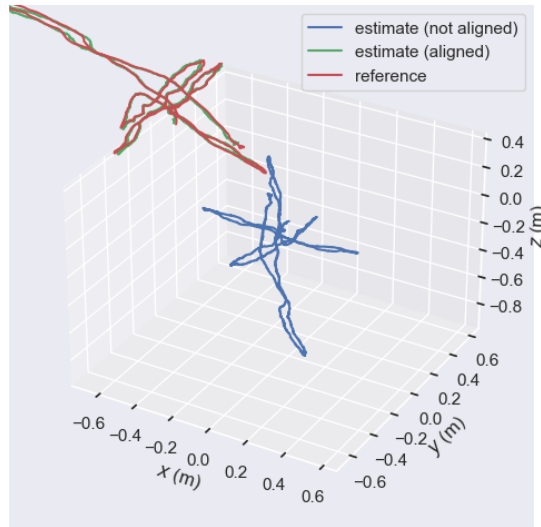


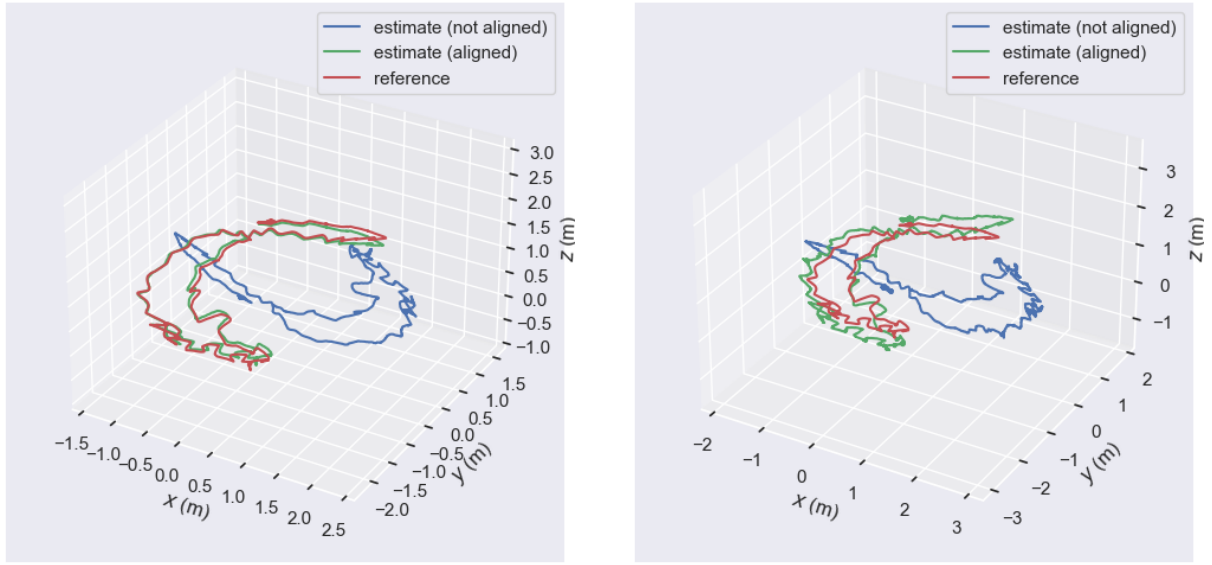
FIGURE 4.2 – Trajectoire estimée et de référence — translations pures (freiburg2_xyz).

Sur la séquence `freiburg2_xyz`, la trajectoire estimée suit fidèlement la référence après alignement, ce qui confirme que DROID-SLAM gère correctement les mouvements simples de translation. Sur `freiburg2_pioneer_slam2`, la trajectoire globale est cohérente mais présente un léger écart en fin de séquence, caractéristique d’une dérive résiduelle sur les longues séquences. Sur `freiburg2_large_checkerboard_calibration`, les résultats sont plus variables en raison des textures répétitives de la cible de calibration, qui perturbent l’établissement de correspondances fiables entre les vues.

4.3 Analyse des paramètres expérimentaux

Les expériences suivantes sont réalisées sur la séquence `freiburg2_desk_with_person` en faisant varier la fréquence d’échantillonnage (30, 10, 5 et 1 fps) et en activant ou non la calibration intrinsèque de la caméra. Pour chaque configuration, les figures présentent la trajectoire estimée comparée à la référence, suivie d’une analyse des résultats.

4.3.1 Essais à 30 fps



(a) Avec calibration

(b) Sans calibration

FIGURE 4.3 – Trajectoires estimée et de référence à 30 fps.

TABLE 4.1 – Métriques APE et RPE à 30 fps.

Configuration	APE moy. (m)	APE RMSE (m)	RPE moy. (m)	RPE RMSE (m)
Avec calibration	0.069	0.082	0.0023	0.0034
Sans calibration	0.263	0.299	0.0048	0.0089

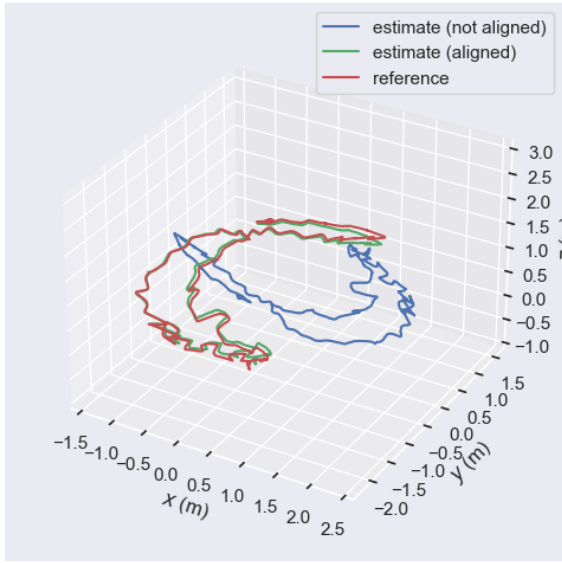
À 30 fps, la calibration réduit l'APE moyenne d'un facteur 4 (de 0.263 m à 0.069 m). La trajectoire estimée avec calibration suit fidèlement la référence, tandis que sans calibration la trajectoire dérive progressivement. La RPE reste faible dans les deux cas, ce qui indique que les déplacements relatifs frame à frame sont bien estimés indépendamment de la calibration.

4.3.2 Essais à 10 fps

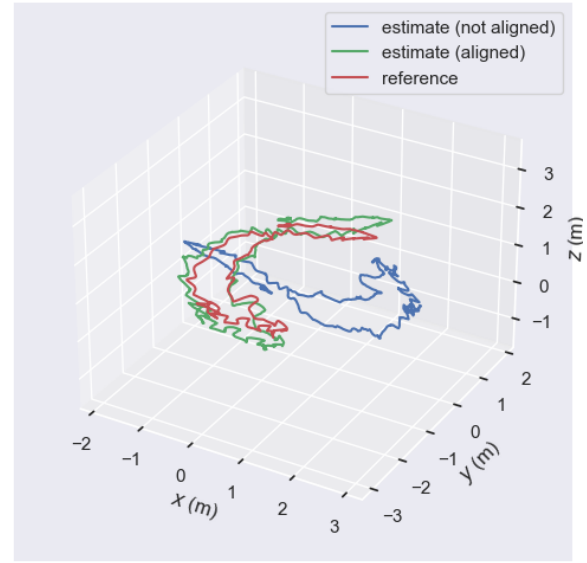
TABLE 4.2 – Métriques APE et RPE à 10 fps.

Configuration	APE moy. (m)	APE RMSE (m)	RPE moy. (m)	RPE RMSE (m)
Avec calibration	0.068	0.082	0.0036	0.0053
Sans calibration	0.263	0.299	0.0086	0.0129

À 10 fps, l'APE avec calibration reste quasi identique à celle obtenue à 30 fps (0.068 m contre 0.069 m), ce qui confirme la robustesse de DROID-SLAM face à la réduction du nombre de frames. La RPE augmente légèrement (de 0.0023 à 0.0036 m), ce qui est attendu : les déplacements entre frames consécutives sont plus grands à fréquence réduite.



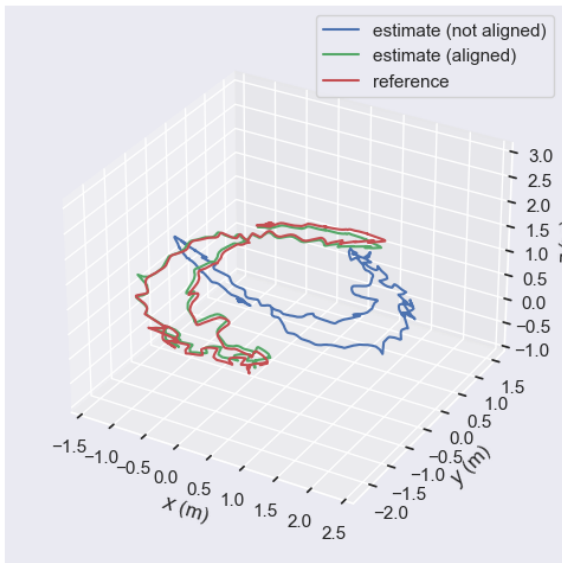
(a) Avec calibration



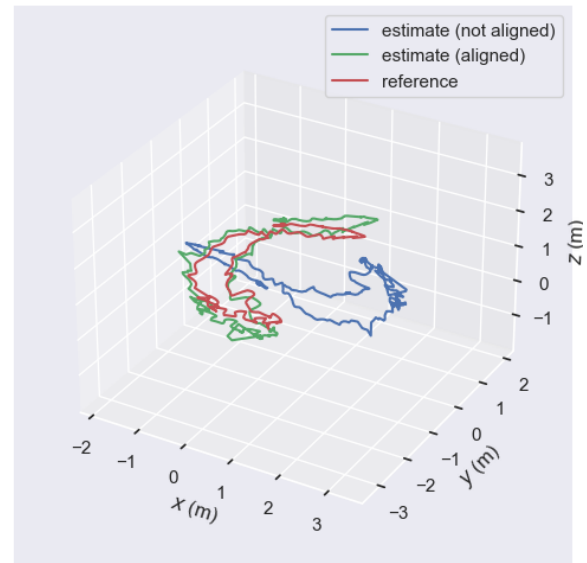
(b) Sans calibration

FIGURE 4.4 – Trajectoires estimée et de référence à 10 fps.

4.3.3 Essais à 5 fps



(a) Avec calibration



(b) Sans calibration

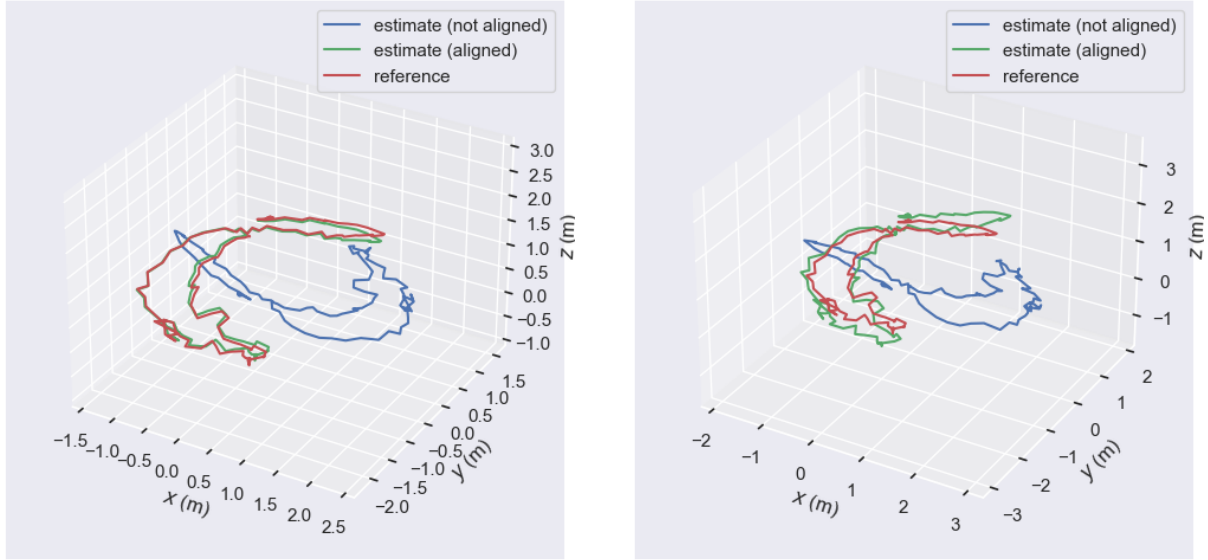
FIGURE 4.5 – Trajectoires estimée et de référence à 5 fps.

À 5 fps, l'APE avec calibration (0.070 m) reste comparable à 30 fps (0.069 m), ce qui montre que la réduction d'un facteur 6 du nombre de frames n'affecte pas la qualité globale de la trajectoire. Cette fréquence représente donc le meilleur compromis entre qualité et efficacité de calcul pour ce type de vidéos.

TABLE 4.3 – Métriques APE et RPE à 5 fps.

Configuration	APE moy. (m)	APE RMSE (m)	RPE moy. (m)	RPE RMSE (m)
Avec calibration	0.070	0.083	0.0047	0.0101
Sans calibration	0.258	0.299	0.0164	0.0290

4.3.4 Essais à 1 fps

**(a)** Avec calibration**(b)** Sans calibration**FIGURE 4.6** – Trajectoires estimée et de référence à 1 fps.**TABLE 4.4** – Métriques APE et RPE à 1 fps.

Configuration	APE moy. (m)	APE RMSE (m)	RPE moy. (m)	RPE RMSE (m)
Avec calibration	0.065	0.078	0.0099	0.0137
Sans calibration	0.260	0.300	0.0325	0.0393

À 1 fps, l'APE avec calibration reste dans le même ordre de grandeur que les fréquences supérieures. Cependant, la RPE se dégrade significativement (0.0099 m contre 0.0047 m à 5 fps), ce qui traduit une estimation moins précise des déplacements locaux lorsque les frames sont trop espacées. Ce résultat confirme que 1 fps est insuffisant pour maintenir une bonne cohérence locale de la trajectoire.

4.4 Synthèse et discussion

4.4.1 Tableau récapitulatif

Le tableau 4.5 récapitule l'ensemble des configurations testées pour une lecture comparative directe.

TABLE 4.5 – Synthèse des métriques APE et RPE selon la fréquence et la calibration — séquence freiburg2_desk_with_person.

Configuration	APE moy. (m)	APE RMSE (m)	RPE moy. (m)	RPE RMSE (m)
30 fps + calibration	0.069	0.082	0.0023	0.0034
30 fps sans calibration	0.263	0.299	0.0048	0.0089
10 fps + calibration	0.068	0.082	0.0036	0.0053
10 fps sans calibration	0.263	0.299	0.0086	0.0129
5 fps + calibration	0.070	0.083	0.0047	0.0101
5 fps sans calibration	0.258	0.299	0.0164	0.0290
1 fps + calibration	0.065	0.078	0.0099	0.0137
1 fps sans calibration	0.260	0.300	0.0325	0.0393

4.4.2 Impact de la calibration

L’observation la plus significative du tableau 4.5 est l’effet déterminant de la calibration sur l’APE. Sans calibration, l’erreur absolue moyenne avoisine 0.26 m quelle que soit la fréquence, contre environ 0.07 m avec calibration, soit un facteur de réduction d’environ 4. Ce résultat s’explique par le fonctionnement de DROID-SLAM : le modèle utilise les paramètres intrinsèques de la caméra (focale et centre optique) pour projeter les correspondances visuelles en coordonnées 3D. Une calibration incorrecte introduit une erreur systématique dans toutes les poses estimées, qui se répercute uniformément sur l’APE.

La RPE est également dégradée sans calibration, mais dans une proportion moindre. La RPE mesurant des déplacements relatifs entre poses consécutives, une erreur systématique constante l’affecte moins que la position absolue.

4.4.3 Impact de la fréquence d’échantillonnage

Avec calibration, l’APE reste stable entre 30 et 1 fps (0.065 à 0.070 m), ce qui témoigne de la robustesse de DROID-SLAM face à la réduction du nombre de frames. En revanche, la RPE augmente progressivement avec la baisse de fréquence, passant de 0.0023 m à 30 fps à 0.0099 m à 1 fps. Ce comportement est attendu : à faible fréquence, le déplacement entre deux frames consécutives est plus important, ce qui rend l’estimation du mouvement relatif plus difficile.

Une fréquence de 5 fps représente le meilleur compromis : l’APE reste comparable à 30 fps (0.070 m contre 0.069 m) tout en réduisant le volume de données d’un facteur 6, ce qui allège significativement la charge de calcul sur le cluster HPC.

4.4.4 Pertinence des métriques pour l’application visée

Au-delà des chiffres, il est important de replacer ces résultats dans le contexte de l’application visée. L’APE mesure la précision de la position absolue de la caméra dans un repère global, ce qui nécessite une trajectoire de référence pour être calculée. Sur les vidéos BabyView, une telle référence n’est pas disponible, ce qui rend cette métrique incalculable en pratique.

La RPE, en revanche, mesure la précision des déplacements relatifs entre frames consécutives. C’est cette information qui présente le plus d’intérêt pour les futures étapes d’apprentissage : l’objectif n’est pas de localiser la caméra dans l’espace absolu, mais de caractériser le mouvement visuel de l’enfant comment sa tête se déplace, dans quelle direction son regard s’oriente, et comment il interagit avec son environnement. Ce sont ces déplacements relatifs qui constituent le signal exploitable pour l’apprentissage multimodal, comme le montrent les travaux d’Aubret et al. [5], où les mouvements de caméra associés aux interactions avec les objets jouent un rôle central dans l’apprentissage de représentations visuelles.

L’APE reste néanmoins utile dans le cadre de ce TER pour valider la cohérence globale du pipeline sur le dataset TUM RGB-D. Les deux métriques sont donc complémentaires : l’APE valide le pipeline, la RPE garantit l’exploitabilité des résultats.

4.4.5 Analyse des erreurs résiduelles

Même dans la meilleure configuration (30 fps avec calibration), une APE moyenne de 0.069 m subsiste. La séquence `freiburg2_desk_with_person` contient une personne en mouvement. DROID-SLAM fait partie des méthodes qui supposent implicitement un environnement statique et ignorent les objets en mouvement, ce qui entraîne des erreurs dans le suivi de la caméra et la reconstruction de la scène [17].

Par ailleurs, l’alignement Umeyama est réalisé sans correction d’échelle (`correct_scale=False`), si bien que toute erreur d’échelle introduite par Metric3D se répercute directement sur l’APE. Ce choix est délibéré : corriger l’échelle masquerait cette source d’erreur et donnerait une image trop optimiste des performances réelles du pipeline. Enfin, les cartes de profondeur produites par Metric3D comportent une incertitude propre qui se propage dans l’initialisation de SLAM, notamment sur les surfaces uniformes ou les zones éloignées de la caméra.

Ces limitations sont inhérentes à toute approche monoculaire sans capteur de profondeur dédié. Les résultats obtenus restent néanmoins cohérents avec les performances attendues sur ce type de séquences, et valident l’utilisation du pipeline sur les vidéos BabyView pour lesquelles une précision centimétrique n’est pas requise.

4.4.6 Temps de calcul et passage à l’échelle

TABLE 4.6 – Temps d’exécution du pipeline pour 5 vidéos (résolution 640×480, durée moyenne 3 min/vidéo) en parallèle sur `hpcgpu2`.

Fréquence	Frames totales	depth.py (min)	slam.py (min)	Total (min)	Moy./vidéo (min)
30 fps	27 000	191	12	203	40.5
10 fps	9 000	42	7	49	9.8
5 fps	4 500	30	5	35	7.0
1 fps	900	24	5	29	5.8

Le module `depth.py` représente entre 83 et 93 % du temps total selon la fréquence, contre 7 à 17 % pour `slam.py`. Ce déséquilibre s'explique par la nature des deux traitements : Metric3D traite chaque image indépendamment, ce qui implique autant de passes réseau que de frames, tandis que DROID-SLAM optimise les poses globalement sur l'ensemble de la séquence.

Entre 30 fps et 5 fps, le temps de `depth.py` est divisé par un facteur 6.4 (de 191 à 30 minutes), cohérent avec la réduction du nombre de frames. Le temps de `slam.py` ne diminue que d'un facteur 2.4 (de 12 à 5 minutes), car la complexité de l'optimisation globale est moins directement proportionnelle au nombre de frames. Le passage de 5 à 1 fps n'apporte qu'un gain de 6 minutes, alors que la RPE se dégrade significativement, ce qui confirme 5 fps comme le meilleur compromis.

Concernant le passage à l'échelle vers BabyView, les vidéos du projet peuvent atteindre 10 à 20 minutes de durée. En extrapolant linéairement depuis les mesures à 5 fps (7 minutes pour 3 minutes de vidéo), le traitement d'une vidéo de 15 minutes représenterait environ 35 minutes de calcul sur `hpcgpu2`. Le parallélisme offert par SLURM permet de traiter plusieurs vidéos simultanément, ce qui rend l'approche viable à l'échelle du projet.

4.5 Validation qualitative sur les vidéos BabyView

4.5.1 Conditions d'application

L'application du pipeline aux vidéos BabyView s'est réalisée dans des conditions particulières liées à la confidentialité des données. L'accès aux vidéos était soumis à des restrictions strictes : il n'était possible que d'exécuter des commandes de traitement sur les fichiers et d'en extraire les résultats, sans pouvoir visualiser directement le contenu des vidéos ni en extraire des images individuelles.

Cette contrainte a rendu impossible l'utilisation des méthodes classiques de calibration intrinsèque. La calibration a donc été réalisée à l'aide de DroidCalib [18], une extension de DROID-SLAM qui estime les paramètres intrinsèques directement à partir du flux vidéo, sans recours à une cible de calibration. Les paramètres obtenus ont ensuite été injectés dans le pipeline pour l'exécution sur les vidéos BabyView.

4.5.2 Protocole de validation

Une validation qualitative a été réalisée sur un échantillon de 10 vidéos BabyView. Pour chacune, le pipeline a été exécuté à 5 fps avec la calibration estimée par DroidCalib, produisant d'une part une trajectoire estimée de la caméra, et d'autre part des cartes de profondeur métrique pour chaque frame.

Deux formes de validation ont été menées. La première consiste à présenter à l'encadrant, Arthur Aubret, les trajectoires estimées sous forme de visualisations 3D et les vidéos correspon-

dantes séparément, sans indiquer quelle trajectoire correspond à quelle vidéo, afin de vérifier si les trajectoires sont suffisamment caractéristiques pour permettre une association correcte. La seconde consiste à inspecter visuellement les cartes de profondeur générées afin de vérifier que la méthode distingue correctement les objets proches des éléments éloignés.

4.5.3 Résultats et analyse

Association trajectoires et vidéos

Sur les 10 vidéos testées, 3 associations ont été correctement établies. Ce résultat, en apparence modeste, s'explique par la nature même des vidéos BabyView et les conditions de l'expérience. Deux catégories de difficultés ont été identifiées.

La première concerne les scènes dans lesquelles l'enfant ne se déplace pas physiquement dans l'espace installé dans un trotteur ou assis à table avec uniquement des mouvements de tête. Dans ce cas, la trajectoire produite se concentre autour d'un point central sans structure spatiale distinctive, et devient visuellement indiscernable d'une vidéo à l'autre. Les figures 4.7 illustrent ce phénomène : les deux trajectoires présentées sont quasi identiques à l'œil nu, bien qu'elles proviennent de vidéos différentes.

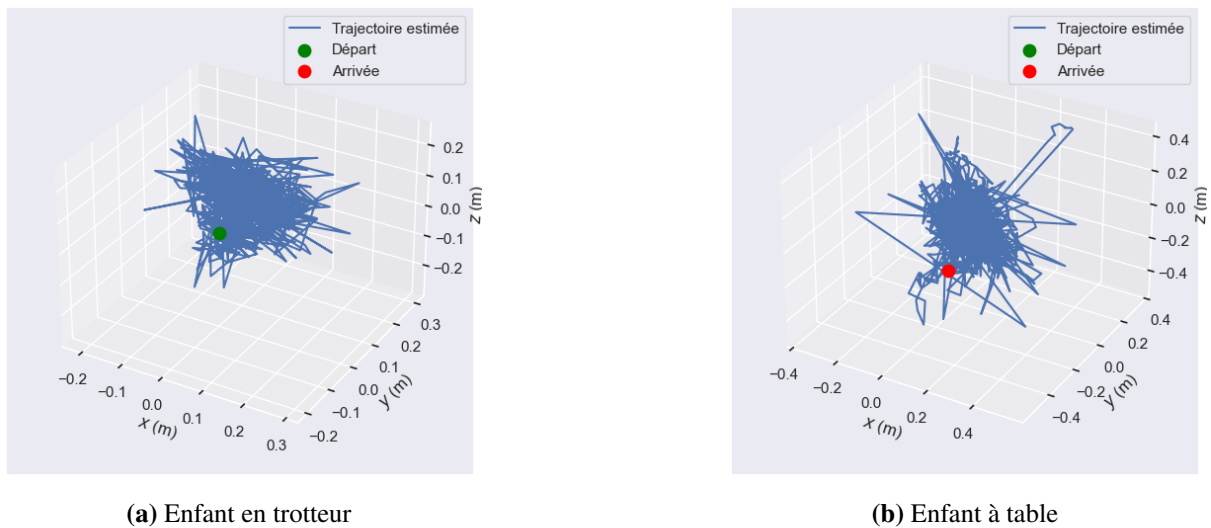


FIGURE 4.7 – Deux trajectoires BabyView issues de scènes sans déplacement global. La similarité visuelle entre les deux rend toute distinction impossible à l'œil nu.

La seconde difficulté concerne les scènes dans lesquelles l’enfant effectue un déplacement dans l’espace puis revient à son point de départ après un certain temps. Dans ce cas, la trajectoire estimée ne se referme pas correctement : au lieu de revenir au point initial, elle présente un écart entre le point de départ et le point d’arrivée. Ce phénomène est une manifestation directe du problème de fermeture de boucle (*loop closure*) inhérent aux méthodes SLAM. Sans mécanisme de reconnaissance des lieux déjà visités, les erreurs s’accumulent frame à frame et ne sont pas corrigées lorsque la caméra revient à un endroit déjà traversé [19]. Comme le soulignent Tian et al. [20], sans détection de boucle, les méthodes SLAM visuelles souffrent d’une dérive non bornée sur les longues séquences. DROID-SLAM ne dispose pas d’un tel module [14] : son optimisation repose sur un Dense Bundle Adjustment récurrent limité à une fenêtre de frames récentes, sans capacité de reconnaissance globale des lieux déjà visités.

Il est également important de noter que la tâche d’association trajectoire–vidéo est intrinsèquement difficile, même pour un observateur humain. Les trajectoires 3D condensent en une visualisation statique l’information de déplacement de toute une vidéo pouvant durer plusieurs minutes, ce qui rend la correspondance avec la vidéo d’origine particulièrement délicate à l’œil nu.

Validation des cartes de profondeur

En parallèle de l’évaluation des trajectoires, les cartes de profondeur métrique générées par Metric3D ont été inspectées visuellement pour chacune des 10 vidéos. Cette inspection a confirmé que la méthode parvient systématiquement à distinguer les objets proches des éléments éloignés de la scène, produisant des cartes cohérentes avec la géométrie attendue des environnements domestiques observés.

En raison des contraintes de confidentialité, il n’est pas possible de présenter des exemples de ces cartes dans ce rapport. Néanmoins, cette validation constitue un indicateur positif quant à la qualité des informations géométriques extraites par le pipeline.

4.5.4 Bilan de la validation

Cette validation qualitative met en évidence deux résultats complémentaires. D’une part, les cartes de profondeur produites sont géométriquement cohérentes et exploitables. D’autre part, les trajectoires estimées sont porteuses d’information pour les séquences dynamiques, mais présentent des limites identifiées pour les scènes statiques et les séquences avec retour au point de départ. Ces limites font l’objet des perspectives présentées dans la section suivante.

4.6 Perspectives d'amélioration

Les résultats obtenus et les limites identifiées au cours de ce TER permettent de dégager plusieurs pistes concrètes pour améliorer le pipeline dans le cadre de travaux futurs.

4.6.1 Intégration d'un mécanisme de fermeture de boucle

Le problème de dérive observé sur les séquences avec retour au point de départ est directement lié à l'absence de module de fermeture de boucle dans DROID-SLAM. Une perspective naturelle est d'intégrer un tel mécanisme, qui permettrait de détecter automatiquement lorsque la caméra repasse par un endroit déjà visité et de corriger la dérive accumulée. Des méthodes récentes basées sur l'apprentissage profond, telles que NetVLAD [20], encodent les images sous forme de descripteurs globaux permettant une reconnaissance efficace des lieux déjà visités, et pourraient être intégrées comme module complémentaire au pipeline actuel.

4.6.2 Amélioration de la calibration intrinsèque

Bien que DroidCalib [18] ait permis de contourner la contrainte de confidentialité, cette méthode ne supporte que les modèles de caméra sans distorsion et nécessite une diversité suffisante de mouvements dans la vidéo pour que les paramètres intrinsèques soient observables. Ces limitations peuvent réduire la précision de la calibration sur des vidéos GoPro, dont l'optique grand angle introduit une distorsion radiale significative.

Une perspective d'amélioration est d'utiliser AnyCalib [21], une méthode publiée à CVPR 2025 qui prédit les paramètres intrinsèques depuis une seule image, de manière indépendante du modèle de caméra. Son approche généralise à des optiques très variées, y compris les grands angles et les caméras fisheye, ce qui la rend particulièrement adaptée aux spécificités de la caméra GoPro utilisée dans BabyView. Intégrer AnyCalib comme étape de pré-traitement avant DROID-SLAM permettrait d'obtenir une calibration plus fiable sans contrainte sur le type de mouvement présent dans la vidéo.

Conclusion

Ce travail avait pour objectif de concevoir et de mettre en œuvre un pipeline permettant d'extraire automatiquement des informations de profondeur métrique et de mouvement de caméra à partir de vidéos égocentriques.

Après avoir étudié plusieurs approches existantes et identifié leurs limites, la solution retenue repose sur la combinaison de Metric3D et DROID-SLAM au travers du pipeline DroidMetric. Sa validation sur le dataset TUM RGB-D a confirmé que la méthode produit des trajectoires cohérentes et exploitables, et que la calibration de la caméra constitue le facteur le plus déterminant pour la qualité des résultats. Le pipeline a ensuite été déployé sur le cluster HPC du Mésocentre UCA et appliqué aux vidéos BabyView, fournissant ainsi les informations géométriques nécessaires aux futures étapes d'apprentissage multimodal.

Au-delà des résultats obtenus, ce TER a été l'occasion de découvrir le fonctionnement d'un laboratoire de recherche et de me confronter à des problématiques scientifiques actuelles en vision par ordinateur. La prise en main d'outils de calcul haute performance, la rigueur exigée dans la mise en place d'un protocole d'évaluation, et les échanges réguliers avec mon encadrant ont consolidé des compétences complémentaires à celles acquises en formation, et ont contribué à développer une vision plus concrète du travail de recherche appliquée.

En perspective, deux pistes principales permettraient d'améliorer le pipeline développé. L'intégration d'un mécanisme de fermeture de boucle permettrait de corriger la dérive observée sur les séquences avec retour au point de départ. Une calibration intrinsèque plus robuste, via une méthode plus fiable, améliorerait également la précision des résultats sur les vidéos GoPro à forte distorsion.

Annexe : Scripts d'évaluation du pipeline

Les scripts présentés ci-dessous ont été développés dans le cadre de ce TER pour préparer et évaluer les trajectoires produites par le pipeline DroidMetric sur le dataset TUM RGB-D, ainsi que pour visualiser les cartes de profondeur générées par Metric3D.

Script `prepare.py` — Préparation des poses au format KITTI

Ce script lit les matrices de transformation produites par DROID-SLAM (une par fichier `.txt` dans le dossier `poses/`), supprime la dernière ligne de chaque matrice 4×4 pour obtenir le format KITTI (3×4), et concatène l'ensemble dans un fichier `joined.txt` utilisé par le script d'évaluation.

Listing 1 – `prepare.py` — Conversion des poses au format KITTI.

```
1 from pathlib import Path
2 import numpy as np
3 import os
4 cwd = os.getcwd()
5 input_dir = os.path.join(cwd, "poses")
6 output_file = "joined.txt"
7 rows = []
8 for file in sorted(Path(input_dir).glob("*.txt")):
9     mat = np.loadtxt(file)
10    mat = mat[:-1] # Suppression de la dernière ligne [0,0,0,1]
11    if mat.shape != (3, 4):
12        raise ValueError(f"{file} n'est pas une matrice 3x4 apres
13                           traitement")
14    rows.append(mat.flatten())
15 np.savetxt(output_file, np.array(rows), fmt="%.18e")
```

Script `eval.py` — Évaluation des trajectoires

Ce script charge les poses estimées depuis `joined.txt` et leur associe les timestamps extraits des noms de fichiers PNG du dossier `rgb/`. Il charge ensuite la trajectoire de référence au format TUM, synchronise les deux trajectoires par association temporelle (tolérance de 50 ms), réalise l'alignement Umeyama sans correction d'échelle, puis calcule les métriques APE et RPE à l'aide de la bibliothèque EVO.

Listing 2 – `eval.py` — Calcul des métriques APE et RPE.

```
1 import os, glob, copy, pprint
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from evo.core.trajectory import PoseTrajectory3D
5 from evo.tools import file_interface
6 from evo.core import sync, metrics
7 from evo.tools import plot
8 from evo.core.units import Unit
9 cwd = os.getcwd()
10 # Timestamps extraits depuis les noms d'images rgb/
11 images_list = sorted(glob.glob(os.path.join(cwd, 'rgb', '*.png')))
12 tstamps = [float(os.path.basename(x)[:4]) for x in images_list]
13 # Chargement des poses estimees (format KITTI)
14 traj_est = file_interface.read_kitti_poses_file(
15     os.path.join(cwd, "joined.txt"))
16 traj_est = PoseTrajectory3D(
17     positions_xyz=traj_est.positions_xyz,
18     orientations_quat_wxyz=traj_est.orientations_quat_wxyz,
19     timestamps=np.array(tstamps))
20 # Chargement du ground truth (format TUM)
21 traj_ref = file_interface.read_tum_trajectory_file(
22     os.path.join(cwd, "groundtruth.txt"))
23 # Association temporelle (tolerance 0.05 s)
24 traj_ref, traj_est = sync.associate_trajectories(
25     traj_ref, traj_est, max_diff=0.05)
26 # Alignement Umeyama sans correction d'echelle
27 traj_est_aligned = copy.deepcopy(traj_est)
28 traj_est_aligned.align(traj_ref,
29     correct_scale=False,
30     correct_only_scale=False)
31 # APE
32 ape_metric = metrics.APE(metrics.PoseRelation.translation_part)
33 ape_metric.process_data((traj_ref, traj_est_aligned))
```

```

34 pprint.pprint(ape_metric.get_all_statistics())
35 # RPE frame a frame
36 rpe_metric = metrics.RPE(
37     pose_relation=metrics.PoseRelation.translation_part,
38     delta=1,
39     delta_unit=Unit.frames,
40     all_pairs=False)
41 rpe_metric.process_data((traj_ref, traj_est))
42 pprint.pprint(rpe_metric.get_all_statistics())
43 # Visualisation
44 fig = plt.figure()
45 plot.trajectories(fig, {
46     "estimee (non alignee)": traj_est,
47     "estimee (alignee)":      traj_est_aligned,
48     "reference":              traj_ref
49 }, plot.PlotMode.xyz)
50 plt.show()

```

Script `profondeur.py` — Visualisation des cartes de profondeur

Ce script parcourt l'ensemble des cartes de profondeur métrique produites par `depth.py` (fichiers `.npy` du dossier `depth/`), normalise chaque carte entre 0 et 1 pour l'affichage, puis les affiche successivement sous forme de *heatmap* afin de vérifier visuellement la cohérence des estimations de profondeur obtenues avec Metric3D.

Listing 3 – `profondeur.py` — Visualisation des cartes de profondeur.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import glob
4
5 # Liste de tous les fichiers .npy
6 files = sorted(glob.glob("depth/*.npy"))
7
8 print(f"{len(files)} cartes trouvees.")
9
10 plt.figure(figsize=(8,6))
11
12 for f in files:
13     depth = np.load(f)

```

```

14
15     # Nettoyage des valeurs invalides
16     depth = np.nan_to_num(depth)
17
18     # Normalisation entre 0 et 1
19     dmin = depth.min()
20     dmax = depth.max()
21
22     if dmax > dmin:
23         depth = (depth - dmin) / (dmax - dmin)
24
25     plt.clf()
26     plt.imshow(depth, cmap="plasma")
27     plt.title(f.split("/")[-1])
28     plt.axis("off")
29
30     plt.pause(0.05)
31
32 plt.show()

```

Bibliographie

- [1] Benjamin Long et al. Babyview : A large-scale egocentric video dataset for understanding early visual experience. *NeurIPS Datasets and Benchmarks*, 2025.
- [2] Kristen Grauman et al. Ego4d : Around the world in 3,000 hours of egocentric video. In *CVPR*, 2022.
- [3] Dima Damen et al. Scaling egocentric vision : The epic-kitchens dataset. In *ECCV*, 2018.
- [4] Jessica Sullivan et al. Headcam : A large-scale dataset of children’s visual experience. *Developmental Science*, 2021.
- [5] Arthur Aubret, Céline Teulière, and Jochen Triesch. Self-supervised visual learning from interactions with objects. In *European Conference on Computer Vision (ECCV)*, pages 54–71. Springer, 2024.
- [6] Rene Ranftl, Katrin Lasinger, David Hafner, Konrad Schindler, and Vladlen Koltun. Towards robust monocular depth estimation : Mixing datasets for zero-shot cross-dataset transfer. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2020.
- [7] David Nistér, Oleg Naroditsky, and James Bergen. Visual odometry. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2004.
- [8] Raul Mur-Artal and Juan D. Tardós. Orb-slam2 : An open-source slam system for monocular, stereo, and rgb-d cameras. *IEEE Transactions on Robotics*, 33(5) :1255–1262, 2017.
- [9] Johannes L. Schönberger and Jan-Michael Frahm. Structure-from-motion revisited. In *CVPR*, 2016.
- [10] Zhang et al. Cusfm : Cuda-accelerated structure-from-motion. *arXiv preprint arXiv :2510.15271*, 2025.
- [11] Jerred Chen and Ronald Clark. Image as an imu : Estimating camera motion from a single motion-blurred image. *arXiv preprint arXiv :2503.17358*, 2025.
- [12] Lihe Yang, Bingyi Kang, Zilong Huang, Zhen Zhao, Xiaogang Xu, Jiashi Feng, and Hengshuang Zhao. Depth anything v2. In *NeurIPS*, 2024.
- [13] Wei Yin, Chi Zhang, Hao Chen, Zhiwei Cai, Gang Yu, Kaixuan Wang, and Chunhua Shen. Metric3d : Towards zero-shot metric 3d prediction from a single image. *International Conference on Computer Vision*, 2024.

- [14] Zachary Teed and Jia Deng. Droid-slam : Deep visual slam for monocular, stereo, and rgb-d cameras. In *NeurIPS*, 2021.
- [15] Jürgen Sturm, Nikolas Engelhard, Felix Endres, Wolfram Burgard, and Daniel Cremers. A benchmark for the evaluation of rgb-d slam systems. *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2012.
- [16] Shinji Umeyama. Least-squares estimation of transformation parameters between two point patterns. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 13(4) :376–380, 1991.
- [17] Moyang Li, Zihan Zhu, Marc Pollefeys, and Daniel Barath. Droid-slam in the wild. *arXiv preprint arXiv :2603.19076*, 2026.
- [18] Annika Hagemann, Moritz Knorr, and Christoph Stiller. Deep geometry-aware camera self-calibration from video. In *International Conference on Computer Vision (ICCV)*, 2023.
- [19] Zhentian Tian et al. Towards accurate loop closure detection in semantic slam with 3d semantic covisibility graphs. *arXiv preprint arXiv :2311.12245*, 2023.
- [20] Hao Tian et al. Real-time loop closure detection in visual slam via netvlad and faiss. *arXiv preprint arXiv :2602.01673*, 2026.
- [21] Alexander Veicht, Paul-Edouard Sarlin, Eric Brachmann, Marc Pollefeys, and Mihai Dusmanu. Anycalib : On-manifold learning for model-agnostic single-view camera calibration. In *Computer Vision and Pattern Recognition (CVPR)*, 2025.